

Static Analysis

Diva Android Application using MobSF

1. What is v1 v2 v3 signature?

Android supports four application signing schemes: v1 scheme based on JAR signing. scheme APK Signature Scheme v2, which was introduced in Android 7.0. v3 scheme APK Signature Scheme v3, which was introduced in Android 9. v4 scheme APK Signature Scheme v3, which was introduced in Android 11.

2. Janus Vulnerability Allows Attackers to Modify Apps without Affecting their Signatures

Android is installed on a huge large number of devices, and that makes it a target for malicious attackers. Vulnerabilities in Google's mobile operating system continue to be discovered every month, but the good news is that Google is usually diligent about fixing them in regular security patches which are then offered to OEMs, who then ship it to devices.

Recently, security researchers uncovered a vulnerability which tricked users into allowing attackers to record their device's screen. That particular exploit was fixed in Android Oreo, but analysts at GuardSquare recently reported another serious vulnerability which affects Android apps signed by an older signature scheme.

GuardSquare's report states that the **Janus vulnerability (CVE-2017-13156) in Android allows attackers to modify the code in applications without affecting their signatures. The report goes onto say that the root of the vulnerability is that a file can be a valid APK file and a valid DEX file at the same time.**

Janus takes advantage of the fact that extra bytes go unnoticed in APK files and DEX files. The GuardSquare report explains that an APK file is a ZIP archive which can contain arbitrary bytes at the start, before and between its ZIP entries. The JAR signature scheme only considers the ZIP entries, ignoring any extra bytes when computing or verifying the application's signature.

It goes on to explain that a DEX file, on the other hand, can contain arbitrary bytes at the end – after the regular sections of strings, classes, method definitions, etc. Therefore, a file can be a valid APK file and a valid DEX file at the same time.

GuardSquare also mentions that a key element of the vulnerability is a **“harmless” feature of the Dalvik/ART virtual machine**. The report states that in theory, the Android runtime loads the APK file, extracts its DEX file and then runs its code. However, in practice, the virtual machine (VM) can load and execute both APK files and DEX files. The issue is that when the VM gets an APK file, it still looks at the magic bytes in the header to decide which type of file it is: DEX or APK. On finding a DEX header, it loads the file as a DEX file. If it doesn't find a header, it loads the file as an APK file containing a zip entry with a DEX file. Thus, it can misinterpret dual DEX/APK files.

GuardSquare says that an attacker can leverage this duality feature of the VM to add a malicious DEX file to a normal APK file without affecting its signature. The Android runtime will accept the APK file as a valid update to a legitimate earlier app version, but the Dalvik VM will load the code from the DEX file, which has been injected with malicious code.

Normally, whenever a user installs an updated version of an app, the app's signature is verified by the Android runtime to ensure that it matches the older version. When the verification is positive, the updated application gets the permissions which had been granted to the original application. In this way, attackers can use the Janus vulnerability to bypass the signature verification process and get unverified code installed on the devices of unsuspecting users.

What's even worse is that this unverified code may get access to powerful permissions. This gives rise to some severe possibilities. GuardSquare states:

“An attacker can replace a trusted application with high privileges (a system app, for instance) by a modified update to abuse its permissions. Depending on the targeted application, this could enable the hacker to access sensitive information stored on the device or even take over the device completely. Alternatively, an attacker can pass a modified clone of a sensitive application as a legitimate update [which] can look and behave like the original application but inject malicious behavior.”

The company added that as of now, they hadn't seen any applications exploiting Janus in the wild. The other bit of good news is the vulnerability requires a user to install a malicious update from a source outside the Google Play Store. Therefore, users who limit app installs to the Play Store are protected.

The Janus vulnerability affects devices running Android 5.0+. Applications that have been signed with APK signature scheme v1 are affected. APKs signed with signature scheme v2 are protected against the vulnerability. This requires that the APKs are running on devices supporting the latest signature scheme (Android 7.0 and newer). Scheme v2 is protected because unlike scheme v1, it considers all bytes in the APK file.

“Older versions of applications and newer applications running on older devices remain susceptible. Developers should at least always apply signature scheme v2,” GuardSquare states.

GuardSquare reported this issue to Google on July 31, 2017, and received acknowledgment on the same day. The company's report says that Google released a patch to its partners in November, and published the bug (CVE-2017-13156) in the Android Security Bulletin on December 4, 2017. The vulnerability has been fixed in the December 2017 Android security patch. Separately, F-Droid applications from their official repository have been confirmed to be safe. Lastly, it has been confirmed that the vulnerability has been patched on APKMirror.

Source : <https://www.xda-developers.com/janus-vulnerability-android-apps/>

3. Debuggable Flag

The android:debuggable attribute defines whether the application can be debugged or not. If an Application is marked as debuggable then an attacker can access the application data by assuming the privileges of that application and can even run arbitrary code under that application permission. In the case of non-debuggable application, attacker would first need to root the device to extract any data. Android applications that are not in the production state are expected to have this attribute set to true to assist the developers however before the actual release of the application this tag should be set to false.

```
<application
```

```
android:debuggable="false"
```

```
</application>
```

Sign your debug build – Debug certificate

When running or debugging your project from the IDE, Android Studio automatically signs your app with a debug certificate generated by the Android SDK tools. The first time you run or debug your project in Android Studio, the IDE automatically creates the debug keystore and certificate in \$HOME/.android/debug.keystore, and sets the keystore and key passwords.

Because the debug certificate is created by the build tools and is insecure by design, most app stores (including the Google Play Store) do not accept apps signed with a debug certificate for publishing.

Android Studio automatically stores your debug signing information in a signing configuration so you do not have to enter it every time you debug. A signing configuration is an object consisting of all of the necessary information to sign your app, including the keystore location, keystore password, key name, and key password. You cannot directly edit the debug signing configuration, but you can configure how you [sign your release build](#).

For more information about how to build and run apps for debugging, see [Build and Run Your App](#).

Expiry of the debug certificate

The self-signed certificate used to sign your app for debugging has an expiration date of 30 years from its creation date. When the certificate expires, you get a build error.

Source:

<https://developer.android.com/studio/publish/app-signing>

4. Android Code Signing Certificate:

Code signing is required for distributing your Android app to Google Play store. It enables to identify who developed the app and ensure that all updates to the app come from you.

5. Browsable Activity:

On Android we can control how the device should react when the user clicks on a link in the web browser, or more specific control which Activity should be started. ... this Activity will start when the user clicks on the third link

The BROWSABLE category revealed

-
- Post published: September 24, 2009
 - Post Author: [Mattias Rosberg](#)
 - Post Category: [Android](#) / [Embedded](#) / [Java](#)

- Post Comments:12 Comments

Assume that you on your desktop computer browse to a web page with the following page source

```
<html>
  <h2>Anywhere on the web.</h2>
  <A HREF="http://www.jayway.com">Jayway</A><br>
  <A HREF="mailto:me@foo.com">Mail Me, Al!</A><br>
  <A HREF="foo://my.test.com">Foo? What's that?</A>
</html>
```

Clicking on the first link you expect your browser to start a new tab and navigate to the url that was specified in the href. If you click on the second link you would expect that the browser opens up your favourite email client. The third link would probably just give you an error message since the browser has no idea how to react on a link like foo://my.test.com

On Android we can control how the device should react when the user clicks on a link in the web browser, or more specific control which Activity should be started. Every time the user clicks on a link the browser sends an intent to the platform with the following content.

action	=	android.intent.action.VIEW
category	=	android.intent.category.BROWSABLE
data	= the text from the href tag	

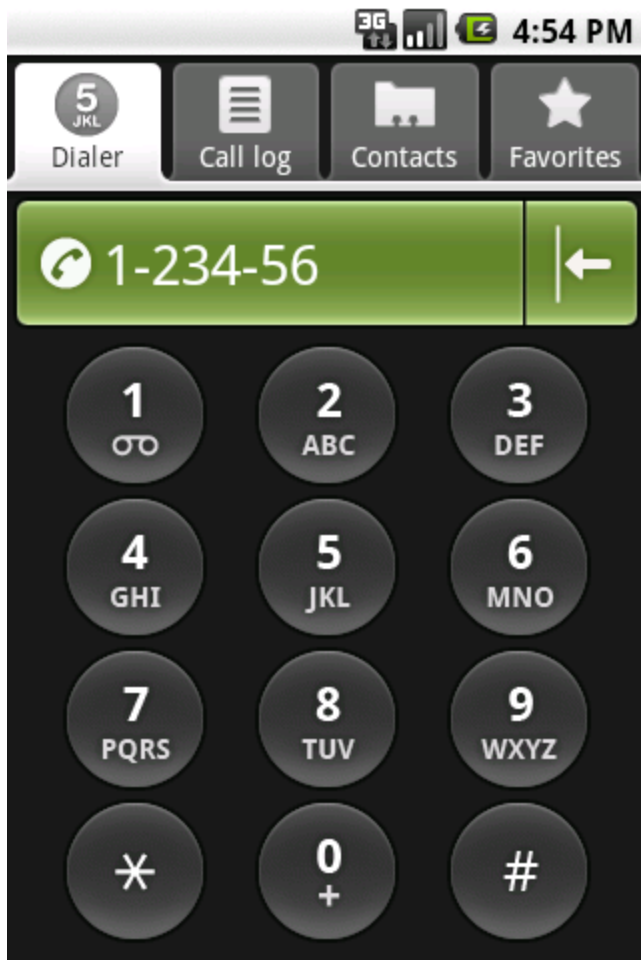
If we navigate to a web page with the following page source on our Andoid device we could expect a few interesting things to happen when we click on the links.

```
<html>
  <h2>Anywhere on the web.</h2>
  <A HREF="tel:123456">Call Me, Al!</A><br>
  <A HREF="mailto:me@foo.com">Mail Me, Al!</A><br>
  <A HREF="foo://my.test.com">Foo? What's that?</A>
</html>
```

The first link will generate an intent with the following information

action	=	android.intent.action.VIEW
category	=	android.intent.category.BROWSABLE
data	=	"tel:123456"

This will start the Dialer application and preset the phone number to 123456.



Clicking on the second link will not surprisingly open up the email client in your device with a preset receiver address. What about clicking on the third link? Well, by default it would just return a “page not available” error message in the browser. The intent sent from the browser will look like this

```
action = android.intent.action.VIEW
category = android.intent.category.BROWSABLE
data = "foo://my.test.com"
```

This means that if we install an Activity with the following IntentFilter

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="foo" android:host="my.test.com" />
</intent-filter>
```

this Activity will start when the user clicks on the third link. Pretty cool, isn't it?

6. Exploiting Debuggable Android Applications

In the previous article, we have seen how to debug Java applications using a little tool called JDB. In this article, we will apply the same logic to exploit Android apps, if they are flagged

as debuggable. If an application is flagged as debuggable, we can inject our own code to execute it in the context of the vulnerable application process.

To make this article more interesting, I have developed a vulnerable application for demonstration purposes, which has a “button” and a “textview”.

Fill out the form below to download the code associated with this article.

If we launch the application, it shows the message “Crack Me”.



Figure 1

If we click the button, it says “Try Again”. Now, our goal is to change the message “Try Again” to “Hacked” without modifying the application’s source code. To be precise, we have to change it at runtime.

Required Tools

- Emulator
- adb – Android Debug Bridge
- jdb – Java Debugger

In my case, to make the installations easier, I am using Android Tamer since all the above required tools are pre-installed.

Topics Involved

- Checking for Vulnerability.
- Getting Ready with the Setup.
- Runtime Code Injection.

Let’s begin the game.

Checking for Vulnerability

In fact, this is the easiest part of the entire article.

1. Decompile the application using APKTOOL to get the AndroidManifest.xml file using the following command.

```
apktool d <vulnerableapp>.apk
```

2. Inspect Androidmanifest.xml file for the following line.

```
android_debuggable=”true”
```

If you find the above line in the AndroidManifest.xml file, the application is debuggable and it can be exploited.

Note: We used APKTOOL to see whether the app is debuggable or not. We won't touch or modify any piece of code as mentioned earlier.

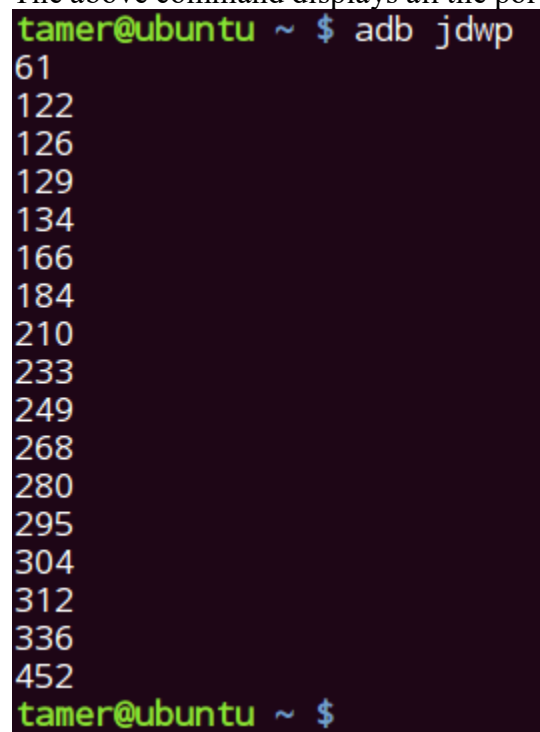
Getting Ready with the Setup

In this step, we will set up all the required things to inject code in to the app during its execution. As mentioned in the previous article, we will use remote debugging in this article.

1. Start Your Emulator
2. Install the vulnerable application
3. Open up your terminal and run the following command to see the Dalvik VM ports listening on the emulator.

`adb jdwp`

The above command displays all the ports on which we can connect and debug as shown below.



```
tamer@ubuntu ~ $ adb jdwp
61
122
126
129
134
166
184
210
233
249
268
280
295
304
312
336
452
tamer@ubuntu ~ $
```

Figure 2

Note: JDWP stands for Java Debug Wire Protocol. If an application running in its VM is debuggable, it exposes a unique port on which we can connect to it using JDB. This is possible in Dalvik Virtual Machines with the support of JDWP.

4. Now, launch our target application and run the same command to see the listening port associated with our target application. It looks as shown below.

```
tamer@ubuntu ~ $ adb jdwp
61
122
126
129
134
166
184
210
233
249
268
280
295
304
312
336
452
543
tamer@ubuntu ~ $
```

Figure 2

If we observe the difference between Figure 2 and Figure 3, there is one extra port 543 listening after launching the target application in Figure 3. We will attach JDB to the application using this port, since this is our target.

- Before attaching to the application, we need to port forward using adb since we are using remote debugging. This is shown in Figure 4.

```
tamer@ubuntu ~ $ adb forward tcp:54321 jdwp:543
tamer@ubuntu ~ $
```

Figure 4

- Now, let's attach JDB to the application as shown in the following figure.

```
tamer@ubuntu ~ $ jdb -attach localhost:54321
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
>
```

Figure 5

Runtime Code Injection

In this step, we will actually exploit the vulnerable application by modifying its behavior at runtime.

To modify the application's behavior at runtime, we need to set up breakpoints and control the flow. But, we don't know what classes and methods are used in the application. So, let's use the following commands to find out the classes and methods used in the application.

To find the classes: "classes"

```
tamer@ubuntu ~ $ jdb -attach localhost:54321
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
> classes
** classes list **
android.R$styleable
android.accounts.Account
android.accounts.Account$1
android.accounts.AccountManager
android.accounts.AccountManager$12
android.accounts.IAccountManager
android.accounts.IAccountManager$Stub
android.accounts.IAccountManager$Stub$Proxy
android.app.Activity
```

Figure 6.1

Since I have too many classes listed, I am listing only a few classes. But if you still scroll down, you will see some interesting user defined classes as shown in the figure below.

```
com.example.debug.MainActivity
com.example.debug.MainActivity$1
```

Figure 6.2

Now, let us see the methods associated with MainActivity\$1 class using the following command.

"methods com.example.debug.MainActivity\$1"

This is shown in figure 7.

```
> methods com.example.debug.MainActivity$1
** methods list **
com.example.debug.MainActivity$1 <init>(com.example.debug.MainActivity)
com.example.debug.MainActivity$1 onClick(android.view.View)
java.lang.Object <init>()
java.lang.Object internalClone(java.lang.Cloneable)
java.lang.Object clone()
java.lang.Object equals(java.lang.Object)
java.lang.Object finalize()
java.lang.Object getClass()
java.lang.Object hashCode()
java.lang.Object notify()
java.lang.Object notifyAll()
java.lang.Object toString()
java.lang.Object wait()
java.lang.Object wait(long)
java.lang.Object wait(long, int)
android.view.View$OnClickListener onClick(android.view.View)
>
```

Figure 7

Now, let's set up a breakpoint at onClick method and control the execution of the app as shown in Figure 8.

"stop in com.example.debug.MainActivity\$1.onClick(android.view.View)"

```
> stop in com.example.debug.MainActivity$1.onClick(android.view.View)
Set breakpoint com.example.debug.MainActivity$1.onClick(android.view.View)
> █
```

Figure 8

To hit the breakpoint, we will have to manually click the button in the application. Once after clicking the button, the breakpoint will be hit and it appears as shown in Figure 9.

```
> stop in com.example.debug.MainActivity$1.onClick(android.view.View)
Set breakpoint com.example.debug.MainActivity$1.onClick(android.view.View)
>
Breakpoint hit: "thread=<1> main", com.example.debug.MainActivity$1.onClick(), line=26 bci=0
<1> main[1] █
```

Figure 9

From here onwards, we will be able to control and see the sensitive values, method arguments, etc. using various commands.

Just to understand what's happening in the background, I am following the code associated with the onClick method, which is shown in Figure 10.

```
btn.setOnClickListener(new OnClickListener() {
    public void onClick(View arg0) {
        // TODO Auto-generated method stub
        TextView tv = (TextView) findViewById(R.id.tvOut);
        Toast.makeText(getApplicationContext(), "Crack Me App For Debugging", Toast.LENGTH_LONG).show();
        String secret = "Try Again";
        tv.setText(secret);
    }
});
```

Figure 10

Before proceeding further, let's see if there are any local variables at this point in time using the command "locals".

```
<1> main[1] locals
Method arguments:
Local variables:
arg0 = instance of android.widget.Button(id=830007782384)
<1> main[1]
```

Figure 11

As we can see, there is no interesting information for us.

So, let's execute the next line using the "next" command as shown below.

```
<1> main[1] next
>
Step completed: "thread=<1> main", com.example.debug.MainActivity$1.onClick(), line=27 bci=11
<1> main[1]
```

Figure 12

Let's again try executing the command "locals" to see what happened in the previous command.

```
<1> main[1] locals
Method arguments:
tv = instance of android.widget.TextView(id=830007784744)
Local variables:
arg0 = instance of android.widget.Button(id=830007782384)
<1> main[1]
```

Figure 13

It's pretty clear that TextView has been loaded into method arguments. If we look at the source code provided in Figure 10, the line associated with TextView instantiation has been executed. Let's now execute the next lines by executing the "next" command, and check the local variables as shown in the figure below.

```
<1> main[1] next
>
Step completed: "thread=<1> main", com.example.debug.MainActivity$1.onClick(), line=29 bci=29

<1> main[1] locals
Method arguments:
tv = instance of android.widget.TextView(id=830007784744)
Local variables:
arg0 = instance of android.widget.Button(id=830007782384)
secret = "Try Again"
```

Figure 14

As we can see, all the local variables have been displayed. The string "secret" looks interesting. The value "Try Again" is what is going to be printed when we click the button.

From Figure 10, it is very clear that the method setText is getting executed to print the value "Try Again". So, let's use the "step" command to get into the definition of the "setText" method and dynamically modify the text to be printed.

```
<1> main[1] step
>
Step completed: "thread=<1> main", android.widget.TextView.setText(), line=2,567 bci=0
<1> main[1]
```

Figure 15

Let's see the local variables inside the definition using "locals".

```
<1> main[1] locals
Method arguments:
Local variables:
text = "Try Again"
<1> main[1]
```

Figure 16

Now, let's change the value of "text" from "Try Again" to "Hacked" using "set" command.

```
<1> main[1] set text = "Hacked"
text = "Hacked" = "Hacked"
<1> main[1]
```

Figure 17

We cannot see any changes in the application, since we haven't executed the modified code.

So, let's run the application using the "run" command as shown below, and see the application's output on its screen.

```
<1> main[1] run
> 
```

Figure 18

Let's look at the application running in the emulator.



Figure 19

We have successfully modified the output of the application at runtime. This is just an example to show how an application's behavior can be modified if the application is debuggable. We can perform various other things including "Getting a shell" on the device in the context of the vulnerable application.

Conclusion

In this article, we have demonstrated how one can exploit an Android application if it is left debuggable when moving it to the production. Pentesters should always look for this vulnerability during their Android app pentests.

Source : <https://resources.infosecinstitute.com/topic/android-hacking-security-part-6-exploiting-debuggable-android-applications/>

7. AllowBackup Flag

The android:allowBackup attribute defines whether application data can be backed up and restored by a user who has enabled usb debugging. If backup flag is set to true, it allows an attacker to take the backup of the application data via adb even if the device is not rooted. Therefore applications that handle and store sensitive information such as card details, passwords etc. should have this setting explicitly set to false because by default it is set to true to prevent such risks.

```
<application  
android:allowBackup="false"  
</application>
```

8. NX Set

The NX bit (no-execute) is a technology used in CPUs to segregate areas of memory for use by either storage of processor instructions (code) or for storage of data, a feature normally only found in Harvard architecture processors.

NX stands for No eXecute and XD stands for eXecute Disable. Both are same and is a technology used in processors to prevent execution of certain types of code.

When disabled, the processor will not restrict code execution in any memory area. This makes the processor more vulnerable to buffer overflow attacks. It is highly recommended that you enable the NX Technology BIOS feature for increased protection against buffer overflow attacks

9. **Stack canaries**, named for their analogy to a canary in a coal mine, are used to detect a stack buffer overflow before execution of malicious code can occur.
10. **RELRO**: Relocation protections. Enabled read only relocations (relro) for statically linked executables and removed all text relocations in Android code. This provides defense in depth against potential memory corruption vulnerabilities.
11. **RPATH**: In computing, rpath designates the run-time search path hard-coded in an executable file or library. In computing, rpath designates the run-time search path hard-coded in an executable file or library. Dynamic linking loaders use the rpath to find required libraries. Specifically, it encodes a path to shared libraries into the header of an executable (or another shared library).
12. **RUNPATH**:
13. **NIAP analysis version 1.3**: The National Information Assurance Partnership (NIAP) was originally created as a partnership between the National Institute of Standards and Technology (NIST) and the National Security Agency (NSA)
NIAP manages a national program for developing Protection Profiles, evaluation methodologies, and policies that will ensure achievable, repeatable, and testable requirements.

https://www.niap-ccevs.org/MMO/PP/-429-M-/pp_app_v1.3_table-reqs.htm

14. APKiD : It gives you information about how an APK was made. It identifies many compilers, packers, obfuscators, and other weird stuff. It's PEiD for Android.

Source : <https://github.com/rednaga/APKiD>

15. Dx compiler : DEX compilation is the process of transforming .class bytecode into .dex bytecode for the Android Runtime (or Dalvik, for older versions of Android).

Tutorial Available at: <https://mobsf.github.io/Mobile-Security-Framework-MobSF/>

<https://www.hacking.reviews/2020/01/mobile-security-framework-mobsf-all-in.html?m=1>